You said:

We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and weight. Each box has internal dimensions, maximum weight capacity, and cost.

Your task is to design and build a small Django-based system that recommends the most suitable box for an order. (this is the context of my project i understand about the project i want to know the requirement and architeture)

ChatGPT said:

You said:

ok it looks complicated but i need to make simple first i want to proceed with logic behind it my logic is to build box recommendation based on dimensions and volume now analyze this give different scenarios and also the limitations i would while proceeding this logic)

ChatGPT said:

You said:

ok i understand about the logic now for database what set of parameters should i mention for the box and also for product . explain logic behind i need to understand

ChatGPT said:

You said:

ok i understand and i build the logic from decimal import Decimal
from inventory.models import Box

```
def calculate_order_totals(order):
    total_weight = Decimal('0')
    total_volume = Decimal('0')
```

```
    for item in order.items.select_related('product').all():
        product = item.product
        quantity = item.quantity

        total_weight += product.weight * quantity

        product_volume = product.length * product.width *
product.height
        total_volume += product_volume * quantity

    return total_weight, total_volume

def product_fits_box(product, box):
    product_dimensions = [product.length, product.width,
product.height]
    box_dimensions = [box.internal_length, box.internal_width,
box.internal_height]

    product_dimensions.sort()
    box_dimensions.sort()

    return all(p <= b for p, b in zip(product_dimensions,
box_dimensions))

def get_suitable_boxes(order):
    total_weight, total_volume = calculate_order_totals(order)
    suitable_boxes = []

    for box in Box.objects.filter(is_active=True):
        if box.max_weight < total_weight:
            continue

        box_volume = box.internal_length * box.internal_width *
box.internal_height

        if box_volume < total_volume:
            continue
```

```
        items = order.items.select_related('product').all()

        if all(product_fits_box(item.product, box) for item in items):
            suitable_boxes.append(box)

    return sorted(suitable_boxes, key=lambda box: box.cost)

def recommend_box(order):
    suitable_boxes = get_suitable_boxes(order)

    if not suitable_boxes:
        return None

    return suitable_boxes[0]( i need to test this )
```

Worked for 26s >

ChatGPT said:

You said:

```
ok all the testcases are working fine from django.http import
JsonResponse
from django.shortcuts import get_object_or_404, render, redirect
from .models import Order, OrderItem
from inventory.models import Product
from .services import recommend_box
import uuid


def recommend_order_box(request, order_id):
    order = get_object_or_404(Order, id=order_id)
    box = recommend_box(order)

    if not box:
        return JsonResponse({'message': 'No suitable box available'},
status=404)

    return JsonResponse({
        'order_number': order.order_number,
        'recommended_box': box.name,
```

```
            'dimensions': {
                'length': float(box.internal_length),
                'width': float(box.internal_width),
                'height': float(box.internal_height)
            },
            'max_weight': float(box.max_weight),
            'cost': float(box.cost)
        })


def place_order(request):
    products = Product.objects.filter(is_active=True)

    if request.method == 'POST':
        customer_name = request.POST.get('customer_name', '').strip()
        selected_items = []

        if not customer_name:
            return render(request, 'products.html', {
                'products': products,
                'error': 'Customer name is required.'
            })

        for product in products:
            try:
                quantity = int(request.POST.get(f'quantity_{product.id}', 0))
            except ValueError:
                return render(request, 'products.html', {
                    'products': products,
                    'error': 'Please enter a valid quantity.'
                })

            if quantity < 0:
                return render(request, 'products.html', {
                    'products': products,
                    'error': 'Quantity cannot be negative.'
                })

            if quantity > 0:
                selected_items.append((product, quantity))

        if not selected_items:
```

```
            return render(request, 'products.html', {
                'products': products,
                'error': 'Please select at least one product.'
            })

        order = Order.objects.create(
            order_number=f"ORD-{uuid.uuid4().hex[:8].upper()}",
            customer_name=customer_name
        )

        for product, quantity in selected_items:
            OrderItem.objects.create(order=order, product=product,
quantity=quantity)

        box = recommend_box(order)

        return render(request, 'order_result.html', {
            'order': order,
            'box': box
        })
```

return redirect('product_list')(i build this one check one for recommendation box and also i build small html interface i created a function place_order when ever user place the order collect all the data and handle the error for no product , no customer name, for negative order_item also)

ChatGPT said:

You said:

i want to test my django view

ChatGPT said:

You said:

ok i almost completed my project i need to test the input validation and edge cases

ChatGPT said:

You said:

prompts which i have used

ChatGPT said:

You said:

no the chat which i made with you

ChatGPT said:

You said:

3. Prompts Used

Some examples of the prompts I used during development:

(fill in this part)

ChatGPT said:

You said:

ok i understand what i extracted information from you

ChatGPT said: